Komikan
Diseño de sincronización y análisis
Martina Veloso y Javier Zavaleta

1. Diseño del esquema de sincronización

El servidor empareja a los clientes y usa un semáforo POSIX para limitar a M las partidas activas. Un mutex protege el estado del asignador de mesas. Cuando se libera un cupo, una pareja en espera puede continuar. Cada partida tiene su propio tablero, su cola FIFO de solicitudes y un hilo juez.

Los trece hilos de fichas esperan un retardo aleatorio configurable entre intentos. Para decidir una jugada, cada ficha obtiene una copia del tablero bajo el mutex de su mesa. Luego evalúa sus opciones sobre esa copia y encola una solicitud. El juez espera con pthread_cond_wait mientras la cola está vacía y la partida sigue activa. Esta llamada libera el mutex durante la espera y lo recupera al despertar. El juez vuelve a comprobar ambas condiciones antes de continuar.

Una solicitud puede quedar desactualizada entre el momento en que la ficha consulta el tablero y el momento en que el juez la atiende. Por eso el juez valida el origen, el destino y las reglas de la jugada sobre el tablero actual. Mantiene el mismo mutex durante la validación y la ejecución. Si dos solicitudes válidas al momento de enviarse apuntan a la misma casilla libre, la primera que la ocupe cambia el tablero. Al procesar la segunda, el juez encuentra el destino ocupado y la rechaza. En una captura, mover al Puma, retirar al perro y actualizar el contador quedan dentro de la misma sección crítica.

Al recibir Ctrl+C, el hilo vigilante solicita el cierre de las mesas activas. Cada mesa cambia su estado bajo su mutex y avisa al juez para que salga de la espera. Las fichas detectan el cierre al consultar la mesa o intentar enviar una solicitud. El hilo de la partida espera con pthread_join antes de destruir la mesa y liberar sus recursos.

2. Riesgos de concurrencia

Deadlock.

Si el hilo encargado del cierre hiciera pthread_join con el candado tomado, quedaría esperando que termine el juez. A su vez, el juez necesitaría recuperar ese candado para comprobar que la partida terminó y salir. Para evitar ese bloqueo, la mesa cambia el estado, despierta al juez con pthread_cond_signal y suelta el candado antes de esperar a los hilos. Además, al usar un candado por mesa, cada jugada requiere adquirir un solo candado para acceder al tablero.

Livelock.

Si muchas fichas vuelven a intentar jugar al mismo tiempo, pueden repetirse los conflictos por casillas y los rechazos de pedidos desactualizados. En ese escenario hay actividad en la cola, pero pocas jugadas llegan a ejecutarse. Los retardos aleatorios espacian los intentos de las fichas y reducen la probabilidad de que vuelvan a competir juntas. Por su parte, el juez comprueba cada pedido sobre el tablero actual para evitar aplicar una jugada que dejó de ser válida.

Starvation.

La cola FIFO conserva el orden de los pedidos que ya entraron. Aun así, una ficha puede pasar mucho tiempo sin moverse si sus destinos se ocupan repetidamente antes de que el juez revise sus pedidos. El orden de atención evita que los pedidos nuevos adelanten a los anteriores, aunque una ficha puede seguir perdiendo la competencia por un destino.

3. Evaluación

Elegimos un candado por mesa porque permite proteger el tablero, la cola y el estado de la partida con un mismo mutex. Esto simplifica la ejecución de una captura, que cambia varias partes del estado. El costo es que el juez ejecuta las jugadas de una misma mesa de a una. Mientras tanto, las fichas pueden buscar jugadas sobre sus copias y los jueces de otras mesas pueden seguir ejecutando jugadas en sus propios tableros.

Validar cada pedido toma tiempo y limita cuántas solicitudes puede procesar el juez por segundo. Si un pedido entra cuando ya hay diez en la cola, el juez debe revisar esos diez primero. Solo los que resulten válidos cambian el tablero. Esta espera aumenta la latencia del pedido nuevo. Cuando la cola está vacía, el juez duerme hasta recibir un aviso, lo que evita que tenga que consultarla continuamente.

Las pruebas de Juez y Mesa comprobaron movimientos, capturas, condiciones de victoria, orden de la cola y cierre del juez con la cola vacía. En la prueba de la casilla compartida se procesaron dos pedidos en orden y se comprobó que se rechazó el segundo. También probamos Ctrl+C durante una partida activa y observamos que el servidor terminó y devolvió el control a la terminal. La evaluación fue funcional, sin mediciones de latencia ni de solicitudes procesadas por segundo.